

Python

Professional Complete Guide

[Syntax](#) · [OOP](#) · [Functional](#) · [Async](#) · [Data Structures](#) · [Decorators](#) · [Type Hints](#) · [Testing](#)

Book 5 of 15 · W3Schools Professional Series · 2026

Table of Contents

#	Chapter	Key Topics
1	Fundamentals	Syntax, types, operators, comprehensions, unpacking
2	Functions Advanced	*args/**kwargs, decorators, closures, functools
3	OOP Complete	Classes, dunder methods, dataclasses, ABC, Protocol
4	Data Structures	list, dict, set, tuple, collections, heapq, deque
5	Functional Python	map/filter/reduce, itertools, functools, generators
6	File I/O & Context Managers	with, pathlib, csv, json, pickle, shelve
7	Exceptions & Logging	try/except/else/finally, custom exceptions, logging
8	Type Hints & Dataclasses	typing, TypeVar, Protocol, @dataclass, pydantic
9	Concurrency	threading, multiprocessing, asyncio, async/await
10	Modules & Packaging	import, __init__, setup.py, pyproject.toml, pip
11	Testing	unittest, pytest, mock, fixtures, parametrize
12	Exercises	20 exercises covering all Python topics

Chapter 1: Fundamentals

Syntax, types, operators, comprehensions

Python Data Types

```
# Numeric types
x: int = 42
y: float = 3.14
z: complex = 2 + 3j
b: bool = True # True == 1, False == 0
n: int = 1_000_000 # underscores for readability

# Strings
s1 = "double quotes"
s2 = 'single quotes'
s3 = """multi
line string"""
s4 = r"raw\nstring" # raw – no escapes
s5 = b"bytes literal"

# f-strings (Python 3.6+)
name, score = "Alice", 98.5
print(f"Name: {name}, Score: {score:.1f}")
print(f"Expr: {2**10 = }") # = for debug (3.8+)
print(f"{name!r}") # repr

# String methods
"hello".strip() # "hello"
"hello world".title() # "Hello World"
"hello".center(11, "-") # "---hello---"
", ".join(["a", "b", "c"]) # "a,b,c"
"a,b,c".split(",") # ["a", "b", "c"]
"hello world".replace("world", "py") # "hello py"
"hello".startswith("he") # True
"hello".find("ll") # 2
"abc".zfill(5) # "00abc"
# f"{name:>10}" right-align to 10 chars
```

Comprehensions & Unpacking

```

# List comprehension
squares = [x**2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
matrix = [[i*j for j in range(1,4)] for i in range(1,4)]

# Dict comprehension
word_len = {word: len(word) for word in ["hi","hello","hey"]}
inverted = {v:k for k,v in {"a":1,"b":2}.items()}

# Set comprehension
unique_lens = {len(w) for w in ["cat","dog","horse","fish"]}

# Generator expression (lazy)
gen = (x**2 for x in range(1_000_000)) # no memory spike
total = sum(x**2 for x in range(1000)) # efficient

# Unpacking
a, b, c = 1, 2, 3
first, *rest = [1, 2, 3, 4, 5] # first=1, rest=[2,3,4,5]
*init, last = [1, 2, 3, 4, 5] # init=[1,2,3,4], last=5
a, b = b, a # swap without temp

# Extended unpacking in for loops
for first, *middle, last in [(1,2,3,4), (5,6,7,8)]:
    print(first, middle, last)

# Walrus operator := (Python 3.8+)
import re
data = "2026-05-10"
if m := re.match(r"(\d{4})-(\d{2})-(\d{2})", data):
    year, month, day = m.groups()

```

Chapter 2: Functions Advanced

Decorators, closures, functools

Decorators

```

import functools, time

# Basic decorator
def timer(func):
    @functools.wraps(func) # preserve metadata
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed= time.perf_counter() - start
        print(f"{func.__name__} took {elapsed:.4f}s")
        return result
    return wrapper

# Decorator with arguments
def retry(times=3, exceptions=(Exception,), delay=1.0):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    if attempt == times - 1: raise
                    time.sleep(delay)
            return wrapper
        return decorator

    @retry(times=3, exceptions=(ConnectionError,), delay=0.5)
    def fetch_data(url: str) -> dict:
        response = requests.get(url)
        response.raise_for_status()
        return response.json()

# Class-based decorator
class validate_types:
    def __init__(self, func):
        self.func = func
        functools.update_wrapper(self, func)
    def __call__(self, *args, **kwargs):
        hints = self.func.__annotations__
        for arg, hint in zip(args, hints.values()):
            if not isinstance(arg, hint):
                raise TypeError(f"Expected {hint}, got {type(arg)}")
        return self.func(*args, **kwargs)

# Stacking decorators
@timer
@retry(times=2)
def slow_network_call(url):
    pass

```

Chapter 3: OOP Complete

Classes, dunder methods, ABC, Protocol, dataclasses

Dunder Methods

```

from __future__ import annotations
from functools import total_ordering

@total_ordering # only need __eq__ and one comparison
class Vector:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    # Representation
    def __repr__(self) -> str:
        return f"Vector({self.x!r}, {self.y!r})"
    def __str__(self) -> str:
        return f"({self.x}, {self.y})"

    # Arithmetic operators
    def __add__(self, other: Vector) -> Vector:
        return Vector(self.x + other.x, self.y + other.y)
    def __sub__(self, other: Vector) -> Vector:
        return Vector(self.x - other.x, self.y - other.y)
    def __mul__(self, scalar: float) -> Vector:
        return Vector(self.x * scalar, self.y * scalar)
    def __rmul__(self, scalar: float) -> Vector:
        return self.__mul__(scalar)
    def __neg__(self) -> Vector:
        return Vector(-self.x, -self.y)

    # Comparison
    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Vector): return NotImplemented
        return self.x == other.x and self.y == other.y
    def __lt__(self, other: Vector) -> bool:
        return abs(self) < abs(other)

    # Numeric protocol
    def __abs__(self) -> float:
        return (self.x**2 + self.y**2) ** 0.5
    def __bool__(self) -> bool:
        return bool(self.x or self.y)

    # Container protocol
    def __len__(self) -> int: return 2
    def __getitem__(self, idx: int) -> float:
        return (self.x, self.y)[idx]
    def __iter__(self):
        yield self.x
        yield self.y

    # Context manager
    def __enter__(self): return self
    def __exit__(self, *args): pass

    # Hashing (needed when __eq__ defined)
    def __hash__(self) -> int:
        return hash((self.x, self.y))

```

Abstract Base Classes & Protocol

```

from abc import ABC, abstractmethod
from typing import Protocol, runtime_checkable

# Abstract Base Class
class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...
    @abstractmethod
    def perimeter(self) -> float: ...

    def describe(self) -> str:
        return f"type(self).__name__: area={self.area():.2f}"

# Protocol (structural subtyping – duck typing with types)
@runtime_checkable
class Drawable(Protocol):
    def draw(self, x: int, y: int) -> None: ...
    def resize(self, factor: float) -> None: ...

# dataclass
from dataclasses import dataclass, field

@dataclass(frozen=True) # immutable
class Point:
    x: float
    y: float
    label: str = ""

@dataclass
class BoundingBox:
    top_left: Point
    bottom_right: Point
    tags: list[str] = field(default_factory=list)

    @property
    def width(self) -> float:
        return self.bottom_right.x - self.top_left.x

    def __post_init__(self):
        if self.width <= 0:
            raise ValueError("width must be positive")

```

Chapter 4: Type Hints & Async

typing module, asyncio, async/await patterns

Type Hints

```
from typing import (
    Any, Optional, Union, Literal,
    List, Dict, Set, Tuple,
    Callable, Iterator, Generator,
    TypeVar, Generic, overload,
    TYPE_CHECKING
)

# Basic hints
def greet(name: str, times: int = 1) -> str:
    return (name + " ") * times

# Optional (value or None)
def find_user(id: int) -> Optional[dict]:
    ...

# Union (Python 3.10+: use X | Y)
def process(value: int | float | str) -> str:
    return str(value)

# TypeVar for generics
T = TypeVar("T")

def first(items: list[T]) -> T | None:
    return items[0] if items else None

# Generic class
class Stack(Generic[T]):
    def __init__(self) -> None:
        self._items: list[T] = []
    def push(self, item: T) -> None:
        self._items.append(item)
    def pop(self) -> T:
        return self._items.pop()

# Callable type
Processor = Callable[[int, str], bool]

# Literal type
Direction = Literal["north", "south", "east", "west"]

# TypedDict
from typing import TypedDict
class UserDict(TypedDict):
    id: int
    name: str
    email: str
```

Async/Await

```
import asyncio

# Basic coroutine
async def fetch_data(url: str) -> dict:
    async with aiohttp.ClientSession() as session:
        async with session.get(url) as response:
            return await response.json()

# Run multiple tasks concurrently
async def main():
    urls = ["https://api.example.com/users",
            "https://api.example.com/posts",
            "https://api.example.com/comments"]

    # gather – run all concurrently
    results = await asyncio.gather(*[fetch_data(url) for url in urls])

    # timeout
    async with asyncio.timeout(5.0):
        data = await fetch_data(urls[0])

    # TaskGroup (Python 3.11+)
    async with asyncio.TaskGroup() as tg:
        t1 = tg.create_task(fetch_data(urls[0]))
        t2 = tg.create_task(fetch_data(urls[1]))
    print(t1.result(), t2.result())

    # Async generator
    async def paginate(url: str):
        page = 1
        while True:
            data = await fetch_data(f"{url}?page={page}")
            if not data: break
            yield data
            page += 1

    async for page in paginate("/api/users"):
        process(page)

    asyncio.run(main())
```

Chapter 5: Exercises

20 professional Python exercises

Exercises

FILL

Ex 1. What does the walrus operator `:=` do?

■ **Answer:** Assigns a value and returns it in a single expression. Example: `if n := len(data): print(n)`. Useful in while loops and comprehensions.

WRITE

Ex 2. Write a decorator that limits function calls per second (rate limiter).

■ **Answer:**

```
import time, functools
def rate_limit(calls_per_sec):
    interval=1.0/calls_per_sec; last=[0.0]
    def dec(f):
        @functools.wraps(f)
        def w(*a, **k):
            now=time.time(); elapsed=now-last[0]
            if elapsed<interval: time.sleep(interval-elapased)
            last[0]=time.time(); return f(*a, **k)
        return w
    return dec
```

TRUEFALSE

Ex 3. Python's list and tuple are both mutable. (True/False)

■ **Answer:** FALSE – list is mutable (can be changed). tuple is immutable (cannot be changed after creation). Both are ordered sequences.

WRITE

Ex 4. Write a context manager class for timing code blocks.

■ **Answer:**

```
import time
class Timer:
    def __enter__(self):
        self.start=time.perf_counter(); return self
    def __exit__(self, *args):
        self.elapsed=time.perf_counter()-self.start
        print(f'Elapsed: {self.elapsed:.4f}s')
```

EXPLAIN

Ex 5. What is the difference between deepcopy and shallow copy?

■ **Answer:** Shallow copy (`copy.copy` or `[:]` or `dict.copy`): creates new container but references same inner objects. Deep copy (`copy.deepcopy`): recursively copies all nested objects. Use deep copy when inner objects must be independent.

WRITE

Ex 6. Write a generic LRU cache decorator.

■ **Answer:** `from functools import lru_cache @lru_cache(maxsize=128) def fib(n): return n if n<2 else fib(n-1)+fib(n-2) # Or from scratch with OrderedDict: from collections import OrderedDict def lru(maxsize): cache=OrderedDict() def dec(f): def w(*a): if a in cache:cache.move_to_end(a);return cache[a] cache[a]=f(*a) if len(cache)>maxsize:cache.popitem(last=False) return cache[a] return w return dec`

FILL

Ex 7. What is the purpose of `__slots__` in a Python class?

■ **Answer:** `__slots__ = ['x','y']` restricts instance attributes to listed names only, reducing memory usage (no `__dict__`) and slightly speeding up attribute access. Useful for classes with many instances.

WRITE

Ex 8. Write a thread-safe singleton class.

■ **Answer:** `import threading class Singleton: _instance=None;_lock=threading.Lock() def __new__(cls): if cls._instance is None: with cls._lock: if cls._instance is None: cls._instance=super().__new__(cls) return cls._instance`

EXPLAIN

Ex 9. What is the GIL and how does it affect Python concurrency?

■ **Answer:** GIL (Global Interpreter Lock) allows only one thread to execute Python bytecode at a time. This prevents true parallelism for CPU-bound tasks in threads. Solutions: multiprocessing (separate processes), asyncio (I/O-bound), `concurrent.futures`, or Cython/C extensions that release the GIL.

WRITE

Ex 10. Write a function that flattens a deeply nested structure.

■ **Answer:** `from collections.abc import Iterable def flatten(nested): for item in nested: if isinstance(item,Iterable) and not isinstance(item,str): yield from flatten(item) else: yield item`

FILL

Ex 11. What does `@property` do in a Python class?

■ **Answer:** Turns a method into a read-only attribute. Use `@prop.setter` for write access. Allows validation and computed attributes with attribute-like syntax.

WRITE

Ex 12. Write a dataclass for a User with validation in `__post_init__`.

■ **Answer:** `@dataclass class User: name:str;email:str;age:int def __post_init__(self): if not self.name:raise ValueError('Name required') if '@' not in self.email:raise ValueError('Invalid email') if not 0<self.age<150:raise ValueError('Invalid age')`

EXPLAIN

Ex 13. What is the difference between iterators and generators?

■ **Answer:** Iterator: any object with `__iter__` and `__next__` methods. Generator: a function using `yield` – automatically creates an iterator. Generators are lazy (compute on demand), memory-efficient. All generators are iterators, not vice versa.

WRITE

Ex 14. Write an async function that fetches multiple URLs concurrently with a semaphore limit.

■ **Answer:**

```
async def fetch_all(urls, limit=5): sem=asyncio.Semaphore(limit) async def fetch_one(url): async with sem: async with aiohttp.ClientSession() as s: async with s.get(url) as r: return await r.json() return await asyncio.gather(*[fetch_one(u) for u in urls])
```

FILL

Ex 15. What is the difference between `is` and `==` in Python?

■ **Answer:** `==` checks value equality. `is` checks identity (same object in memory). Use `==` for comparing values; `is` only for `None` checks (`x is None`) or singletons.

WRITE

Ex 16. Write a metaclass that automatically registers all subclasses.

■ **Answer:**

```
class Registry(type): _registry={} def __init_subclass__(cls, **kw): super().__init_subclass__(**kw) Registry._registry[cls.__name__]=cls class Base(metaclass=Registry): pass class Dog(Base): pass class Cat(Base): pass print(Registry._registry) # {Dog:cls, Cat:cls}
```

EXPLAIN

Ex 17. What is the purpose of `__enter__` and `__exit__` in context managers?

■ **Answer:** `__enter__`: called at start of 'with' block, returns value bound to 'as' variable. `__exit__(exc_type, exc_val, tb)`: called at end, even if exception. Return `True` to suppress the exception. Used for resource management (files, locks, db connections).

WRITE

Ex 18. Write a function that groups a list of dicts by a key.

■ **Answer:**

```
from itertools import groupby from collections import defaultdict def group_by(items, key): result=defaultdict(list) for item in items: result[item[key]].append(item) return dict(result)
```

TRUEFALSE

Ex 19. Python's dict maintains insertion order as of Python 3.7. (True/False)

■ **Answer:** `TRUE` – As of Python 3.7 (CPython 3.6), dicts maintain insertion order as a language specification, not just an implementation detail.

WRITE

Ex 20. Write a complete implementation of a min-heap using a list.

```
■ Answer: import heapq class MinHeap: def __init__(self):self._h=[] def
push(self,v):heapq.heappush(self._h,v) def pop(self):return heapq.heappop(self._h) def
peek(self):return self._h[0] def __len__(self):return len(self._h) def
pushpop(self,v):return heapq.heappushpop(self._h,v)
```